Birla Institute of Technology & Science, Pilani

Work Integrated Learning Programmes Division

AIML* ZG523 · MLOps

Lab Experiment 2

Containerize and Orchestrate a Model Deployment Pipeline (Docker + Airflow)

Module / Sessions	M2–M3 · Contact Sessions 5–6 (Part A due after S5, Part B after S6)
Mapped Outcomes	LO2 (CO2)
Builds on	Lab 1 — requires your cartvista-mlops folder with mlflow.db and the @champion model
Mode	Self-paced. Internet needed once for Docker/Airflow installs; runs offline thereafter
Estimated Effort	3 – 3.5 hours (A: ~1.5 h, B: ~2 h)
Readings	T2 Ch. 7 (Deployment); T1 Ch. 5–6

1. Lab Storyline — "It Works on My Laptop"

Your Lab 1 work was a hit. The retention team asked Priya to score last week's customers with the champion model — and it crashed on her machine with a scikit-learn version error. Her laptop has sklearn 1.2; you trained on 1.5. Two hours were lost proving the model wasn't broken, the environment was.

Your team lead makes two decisions. First: the model runs inside a Docker container from now on, so the environment travels with it — that is Part A. Second: the weekly retrain-validate-register cycle you have been running by hand becomes an orchestrated pipeline that runs the same way every time, with a quality gate so a bad model can never silently become champion — that is Part B, with Apache Airflow as the orchestrator.

Continuity: You are packaging the exact @champion from Lab 1 and automating the exact scripts you wrote there. Nothing is thrown away; it is industrialized.

2. Objective

To package the CartVista churn model as a self-contained Docker image that scores customer batches identically on any machine, and to orchestrate the train → validate → register lifecycle as an Airflow DAG with an explicit quality gate.

3. Learning Outcomes (Summarized)

- Export a registered model and build a runnable Docker image around it, with pinned dependencies.
- Explain what an image, container, layer and volume are — by having used each one.
- Express an ML lifecycle as a DAG with task dependencies, retries, and a validation gate.
- Reason about idempotency: why every task must be safe to re-run.

4. Prerequisites

- Completed Lab 1 (the cartvista-mlops folder with mlflow.db, churn_data.csv, and registered @champion).
- Docker Desktop (Windows/macOS) or Docker Engine (Linux) installed and running: verify with `docker run hello-world`
- Part B needs Airflow, which does not run natively on Windows — Windows students use WSL2 (Ubuntu) and work inside it. macOS/Linux proceed directly.

Windows + WSL2: Install Ubuntu from the Microsoft Store, open its terminal, and re-create the venv there (cd to your project via /mnt/c/...). Docker Desktop integrates with WSL2 automatically (Settings → Resources → WSL Integration).

5. Part A — Containerize the Champion

Step A1 — Export the champion out of the registry

A container should not need your mlflow.db. This script copies the champion's artifacts into a plain folder the image can bake in.

export_champion.py

```
"""Export @champion artifacts to ./model_export for Docker packaging."""
import shutil, pathlib, mlflow
from mlflow import MlflowClient

mlflow.set_tracking_uri("sqlite:///mlflow.db")
client = MlflowClient()
mv = client.get_model_version_by_alias("cartvista-churn", "champion")
print(f"Exporting cartvista-churn v{mv.version} (run {mv.run_id[:8]})")

dst = pathlib.Path("model_export")
if dst.exists(): shutil.rmtree(dst)
local = mlflow.artifacts.download_artifacts(
    artifact_uri=f"runs:/{mv.run_id}/model", dst_path=".")
shutil.move(local, dst)
print(f"Model exported to {dst}/ -- contents: {[p.name for p in dst.iterdir()]}")
```

terminal

```
python export_champion.py
Exporting cartvista-churn v1 (run ab6602cd)
Model exported to model_export/ -- contents: ['MLmodel', 'model.pkl',
  'conda.yaml', 'python_env.yaml', 'requirements.txt']
```

Expected outcome: a model_export folder. Open requirements.txt inside it — MLflow recorded the exact library versions of your training run. That file is about to solve Priya's problem.

Step A2 — The batch scorer the container will run

score_batch.py

```
"""Container entrypoint: score a CSV of customers with the baked-in model."""
import sys, pandas as pd, mlflow.pyfunc

IN_PATH  = sys.argv[1] if len(sys.argv) > 1 else "/data/input.csv"
OUT_PATH = sys.argv[2] if len(sys.argv) > 2 else "/data/scored.csv"

model = mlflow.pyfunc.load_model("/app/model_export")
df = pd.read_csv(IN_PATH)
features = df.drop(columns=["churn"], errors="ignore")
df["churn_pred"] = model.predict(features)
df.to_csv(OUT_PATH, index=False)
print(f"Scored {len(df)} customers -> {OUT_PATH} "
      f"(predicted churners: {int(df.churn_pred.sum())})")
```

Step A3 — Write the Dockerfile and build the image

Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

# Install EXACTLY what training used -- this is the whole point
COPY model_export/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY model_export/ /app/model_export/
COPY score_batch.py .

ENTRYPOINT ["python", "score_batch.py"]
```

terminal

```
docker build -t cartvista-scorer:v1 .
docker images cartvista-scorer
```

```
REPOSITORY          TAG   IMAGE ID       CREATED          SIZE
cartvista-scorer    v1    3c1f9a2b8d44   8 seconds ago    612MB
```

Expected outcome: an image around 550–700 MB. Note the layer caching on a second build — only changed layers rebuild. Exercise E2 will make you shrink this image.

Step A4 — Run it, with a volume for data

The image contains code + model + environment. Data stays outside and is mounted in — the separation that makes one image reusable for every week's batch.

```
terminal
# from the cartvista-mlops folder:
docker run --rm -v "$(pwd):/data" cartvista-scorer:v1 /data/churn_data.csv /data/scored.csv
# Windows PowerShell:  -v "${PWD}:/data"
Scored 5000 customers -> /data/scored.csv (predicted churners: 231)
```

Expected outcome: scored.csv appears in your project folder with a churn_pred column. Your churner count may differ by a few. This container now runs identically on Priya's laptop, a CI server, or a cloud VM — Part A complete.

6. Part B — Orchestrate the Lifecycle with Airflow

Step B1 — Install and start Airflow (WSL2/macOS/Linux)

terminal

```
pip install "apache-airflow==2.9.2" --constraint \
  "https://raw.githubusercontent.com/apache/airflow/constraints-2.9.2/constraints-3.11.txt"

export AIRFLOW_HOME=$(pwd)/airflow_home
airflow standalone     # first run creates the DB and an admin user
standalone | Airflow is ready
standalone | Login with username: admin  password: <printed here>
```

Expected outcome: UI at http://localhost:8080. Note the admin password from the console (also in airflow_home/standalone_admin_password.txt). Keep this terminal running; open a second one for the next steps (re-export AIRFLOW_HOME there too).

Step B2 — The pipeline DAG with a quality gate

Create the dags folder and the DAG file. The DAG automates your Lab 1 loop: refresh data → train two candidates → validate the best against a quality bar → only then promote it to champion. The gate task is the culture change: no human can skip validation, because promotion is downstream of it.

airflow_home/dags/cartvista_pipeline.py

```
"""CartVista weekly retrain pipeline: data -> train x2 -> gate -> promote."""
from datetime import datetime
import subprocess, json, pathlib
from airflow.decorators import dag, task

PROJECT = pathlib.Path(__file__).resolve().parents[2]   # cartvista-mlops
AUC_GATE = 0.72          # a model below this bar must NOT become champion

def run(cmd):
    r = subprocess.run(cmd, cwd=PROJECT, shell=True,
                       capture_output=True, text=True)
    print(r.stdout, r.stderr)
    r.check_returncode()
    return r.stdout

@dag(schedule=None, start_date=datetime(2026, 1, 1), catchup=False,
     tags=["cartvista"])
def cartvista_weekly_retrain():

    @task
    def refresh_data():
        run("python generate_data.py")

    @task
    def train_logreg():
        run("python train_tracked.py --model logreg --C 1.0")

    @task
    def train_rf():
        run("python train_tracked.py --model rf --n-estimators 200 --max-depth 8")

    @task
    def validate_best() -> float:
        import mlflow
        from mlflow import MlflowClient
        mlflow.set_tracking_uri(f"sqlite:///{PROJECT}/mlflow.db")
        client = MlflowClient()
        exp = client.get_experiment_by_name("cartvista-churn")
        best = client.search_runs([exp.experiment_id],
                order_by=["metrics.test_auc DESC"], max_results=1)[0]
        auc = best.data.metrics["test_auc"]
        print(f"Best AUC this cycle: {auc:.4f} (gate: {AUC_GATE})")
        if auc < AUC_GATE:
            raise ValueError(f"Quality gate FAILED: {auc:.4f} < {AUC_GATE}")
        return auc
```

```
    @task
    def promote():
        run("python register_model.py")

    d = refresh_data()
    gate = validate_best()
    d >> [train_logreg(), train_rf()] >> gate >> promote()

cartvista_weekly_retrain()
```

Step B3 — Trigger, watch, and read the run

```
  terminal (second window, AIRFLOW_HOME exported)
airflow dags trigger cartvista_weekly_retrain
```

In the UI open the DAG → Grid view. Watch tasks turn from running (green-ish) to success (dark green): refresh_data first, then the two training tasks in parallel, then validate_best, then promote. Click validate_best → Logs and find the printed AUC-vs-gate line. Then open the Graph view and confirm the diamond shape: one source, parallel middle, one gate, one sink.

Expected outcome: All five tasks succeed; MLflow's Models page shows a new version of cartvista-churn with @champion moved to it. You have replaced a human ritual with a pipeline — and the pipeline refuses to promote junk.

Step B4 — Prove the gate works

Edit the DAG: set AUC_GATE = 0.95 (an impossible bar), save, wait ~30 s for Airflow to re-parse, and trigger again.

```
validate_best   FAILED   ValueError: Quality gate FAILED: 0.7561 < 0.95
promote         upstream_failed  (never ran)
```

Expected outcome: the run goes red at the gate and promote is skipped — the champion alias did NOT move. Screenshot this Grid view; it is a required deliverable. Restore AUC_GATE = 0.72 afterwards.

7. Exercises

E1 — Retry policy (10 min)

Add retries=2 and a 30-second retry_delay to the training tasks (see @task parameters). Why are retries safe for these tasks but potentially dangerous for a task that, say, sends retention emails? Two sentences on idempotency.

E2 — Shrink the image (20 min)

The scorer image is ~600 MB. Reduce it below 450 MB without breaking Step A4 — hints: do you need all of the exported requirements at inference time? Report before/after sizes from docker images and what you removed.

E3 — Schedule it (10 min)

Change schedule=None to a cron expression that runs every Monday at 02:00. Show the next three run dates the UI computes. Why is catchup=False the right setting here?

E4 — Containerized task (15 min)

Add a final score_weekly task to the DAG that runs your Part A container via `docker run` on the freshly promoted champion (re-export first!). This closes the loop: the pipeline now ends with a business artifact, not just a registry entry.

E5 (stretch) — Failure alert (15 min)

Give validate_best an on_failure_callback that writes gate_failures.log with a timestamp and the failing AUC. Trigger with the impossible gate to test it.

8. Troubleshooting

Symptom	Fix
`docker: permission denied`	Linux: add yourself to the docker group (sudo usermod -aG docker $USER) and re-login. Windows/macOS: ensure Docker Desktop is running.
Airflow UI empty / DAG missing	AIRFLOW_HOME not exported in this terminal, or the file has a syntax error — run `python airflow_home/dags/cartvista_pipeline.py` to check, then wait 30 s.
Port 8080 already in use	airflow standalone respects AIRFLOW__WEBSERVER__WEB_SERVER_PORT=8081 (export before starting).
Tasks fail with 'mlflow.db not found'	The DAG resolves PROJECT relative to the dags folder — airflow_home must live INSIDE cartvista-mlops as shown, or adjust PROJECT.
Windows: pip install airflow fails	Expected — Airflow does not support native Windows. Do all of Part B inside WSL2 Ubuntu.
Volume mount shows empty /data	Quote the mount and use the shell-correct pwd form from Step A4; on Docker Desktop confirm the drive is shared (Settings → Resources → File sharing).

9. Deliverables Checklist (for EC-1 Lab Assignment)

- Terminal output of the Step A4 container run, plus `docker images` showing your tag.
- Screenshot: Airflow Grid view of one fully green run (Step B3).
- Screenshot: the red gate-failure run with promote showing upstream_failed (Step B4).
- Written answers: E1 (idempotency), E2 (before/after sizes + method), E3 (cron + catchup reasoning).
- E4 DAG code if attempted.

- Reflection: “The gate stopped a bad model from being promoted. What can it NOT stop?” (Hint: think about the data itself — this is the doorway to Lab 4.)

Academic integrity: Docker image IDs, Airflow run timestamps and log excerpts are machine-unique and are checked across submissions.